



-Paradigma Orientado a Objetos-

**Trabajo Práctico Integrador (UVA 1 y UVA 2):
Arquitectura del Sistema de Control Sky-Delivery
S.A.**

Integrantes

1178898 | Manno, Santino

1182237 | Panessi, Selena

1202072 | Bonata, Maria del Rosario

1191037 | Bramuglia, Maximo

1201842 | Seoane, Alejo

1142008 | Cuadrado Tomás

1183586 | Luciana Pauluk

1179132 | Gabriel Martinez

Profesor: Pablo Eduardo Hernandez

Turno: Jueves-Virtual (1er Cuatrimestre - 2026)

2. Desafío de Modelado UML (UVA 1)

2.1 Identificación de Componentes y los 3 Pilares

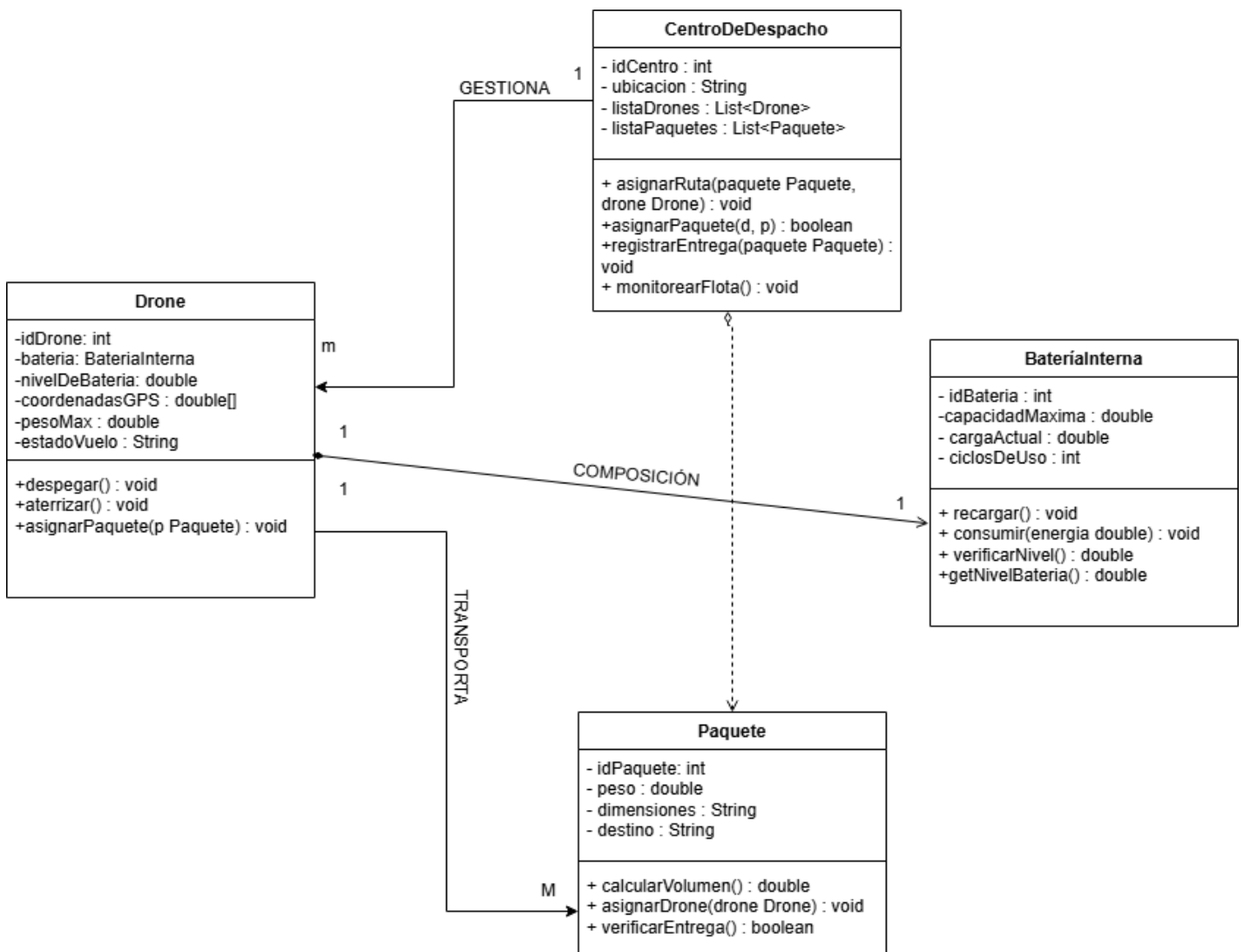
- **Drone**
 - **Estado:** idDrone (int), bateria (BateriaInterna), nivelDeBateria (double), coordenadasGPS (double[]), pesoMax (double), estadoVuelo (String).
 - **Comportamiento:** despegar(), aterrizar(), asignarPaquete(Paquete p).
 - **Identidad:** Se define mediante el atributo idDrone.
- **Paquete**
 - **Estado:** IdPaquete (int), peso (double), dimensiones (String), destino (String).
 - **Comportamiento:** obtenerPeso(), asignarDestino(), verificarDatos().
 - **Identidad:** se define por idPaquete.
- **CentroDeDespacho**
 - **Estado:** idCentro (int), ubicacion (String), listaDrones (List<Drone>), listaPaquetes (List<Paquete>).
 - **Comportamiento:** asignarRuta(paquete Paquete, drone Drone), asignarPaquete(d, p), registrarEntrega(paquete Paquete), monitorearFlota().
 - **Identidad:** Se define mediante idCentro.
- **BateriaInterna**
 - **Estado:** IdBateria(int), capacidadMaxima (double), cargaActual (double), ciclosDeUso (int).
 - **Comportamiento:** recargar(), consumir(energia double), verificarNivel(), getNivelBateria().
 - **Identidad:** Se define mediante idBateria.

2.2 Relaciones y Preservación de la Integridad

- **Relación Drone-Paquete:** El drone transporta el paquete, pero ambos pueden existir por separado.
 - **Cardinalidad:** 1 → M
 - **Direccionalidad:** Drone → Paquete
 - **Tipo:** Asociación
- **Relación Drone-BateriaInterna:** La batería depende totalmente del drone. Si el drone se elimina, la batería también.
 - **Cardinalidad:** 1 → 1
 - **Direccionalidad:** Drone → BateriaInterna
 - **Tipo:** Composición

- **Relación CentroDeDespacho – Drone:** El centro administra múltiples drones.
 - **Cardinalidad:** 1 → M
 - **Direccionalidad:** CentroDeDespacho → Drone
 - **Tipo:** Asociación
- **Relación CentroDeDespacho – Paquete:** El centro gestiona los paquetes (los asigna a drones), por lo que mantiene control sobre ellos.
 - **Cardinalidad:** 1 → M
 - **Direccionalidad:** CentroDeDespacho → Paquete
 - **Tipo:** Dependencia.

2.3 Diagrama de Clases UML



3. Desafíos Técnicos de Arquitectura y Memoria (UVA 2)

3.1 Gestión en la Máquina Virtual (VM)

- I. **Segmentación de Memoria:** Explique por qué las variables de referencia de un Drone se almacenan en el Stack , mientras que la instancia física del objeto reside en el Heap . Detalle qué ocurre con dicha memoria cuando el objeto pierde su alcance (scope).

Las variables de referencia de un Drone se almacenan en el Stack porque son variables locales. Estas contienen la dirección de memoria del objeto.

La instancia real del Drone se almacena en el Heap, ya que es un objeto dinámico creado en tiempo de ejecución mediante new.

Cuando el objeto pierde su alcance (scope) ya no hay referencias que lo apunten:

- La referencia en el Stack se elimina automáticamente al salir del método.
- El objeto en el Heap queda sin referencias.
- El Garbage Collector de la VM lo elimina posteriormente para liberar memoria.

- II. **Ciclo de Vida VM:** Describa el proceso de post-procesamiento, desde la compilación del código fuente hasta la ejecución de instrucciones por la VM para este sistema logístico.

El ciclo de vida en la Máquina Virtual empieza cuando el desarrollador escribe el código fuente en Java. Se debe compilar utilizando el compilador (javac), que lo transforma en bytecode (.class), un formato intermedio independiente del sistema operativo.

Una vez generado el bytecode, la Java Virtual Machine se encarga de procesarlo. En primer lugar, el Class Loader carga las clases necesarias en memoria. Luego, el bytecode es verificado para garantizar que sea seguro y que no contenga errores que puedan afectar la ejecución.

Posteriormente, el Execution Engine ejecuta el programa. Esto puede hacerse interpretando el bytecode o compilándolo a código nativo en tiempo de ejecución mediante el compilador JIT (Just-In-Time), lo que permite mejorar el rendimiento.

Durante toda la ejecución, la Java Virtual Machine administra automáticamente la memoria: los objetos se crean en el Heap, las variables de referencia se manejan en el Stack y el Garbage Collector se encarga de liberar la memoria de los objetos que ya no están en uso.

3.2 Tipos de Datos: Primitivo vs. Wrapper

Para el atributo `nivelDeBateria`, justifique la elección entre un tipo de dato primitivo (`double`) o un tipo de dato wrapper (`Double`).

Se recomienda el uso de primitivo (`double`) cuando el rendimiento y el uso eficiente de memoria son críticos, como en el procesamiento de telemetría en tiempo real. Además, al tratarse de un valor numérico que representa el nivel de batería, no es necesario utilizar métodos adicionales que ofrecen las clases wrapper.

Se recomienda el wrapper (`Double`) si se requiere que el atributo pueda ser null (ej. batería no detectada) o si se almacenará en colecciones de Java (como `ArrayList`).

3.3 Estructuras de Control y Manejo de Arreglos

- Firma y Contrato:

```
public boolean autorizarVuelo() {  
  
    //Obtener batería actual  
  
    double nivel = bateria.getNivelBateria();  
  
    //Obtener el peso del paquete que el drone va a  
llevar  
  
    double peso = paquete.getPeso();  
  
    //Validar autonomía  
  
    if (nivel >= peso * 0.5) { //para condicionar el  
consumo de energía de la batería por el peso/volumen que  
tiene el paquete  
  
        return true;  
  
        System.out.println("autorizado");  
  
    } else {  
  
        return false;  
  
        System.out.println("No autorizado");  
  
    }  
  
}
```

- Instanciación de Flota:

```
Drone[] flota = new Drone[50];

for (int i = 0; i < flota.length; i++) {

    flota[i] = new Drone(i, new BateriaInterna(), 100.0 , new
double[]{0.0,0.0}, 10.0, "En tierra");

}

//cada Drone recibe un idDrone único (identificación única),
se instancia su batería interna y se inicializan sus
atributos básicos que pusimos en el UML para que se pueda
ejecutar.
```

- Nota crítica:

- Al declarar, creamos el arreglo con 50 posiciones, pero cada posición contiene un objeto null. Básicamente espacios vacíos, no drones reales.

```
Drone[] flota = new Drone[50];
```

- Luego con la instancia individual, se crean 50 objetos Drone en el Heap, y cada referencia del arreglo apunta a un objeto válido.

```
for (int i = 0; i < flota.length; i++) {

    flota[i] = new Drone(i, new BateriaInterna(), 100.0 ,
new double[]{0.0,0.0}, 10.0, "En tierra");

}
```

- Si intentas invocar un método sobre tu referencia null, un ejemplo en este caso (flota[0].despegar), la VM lanza un NullPointerException porque no hay un objeto en tu memoria. El instanciar te asegura que cada elemento se pueda ejecutar gracias a su estado inicial (en conjunto a sus métodos)

4. Integración Dinámica: Diagrama de Secuencia

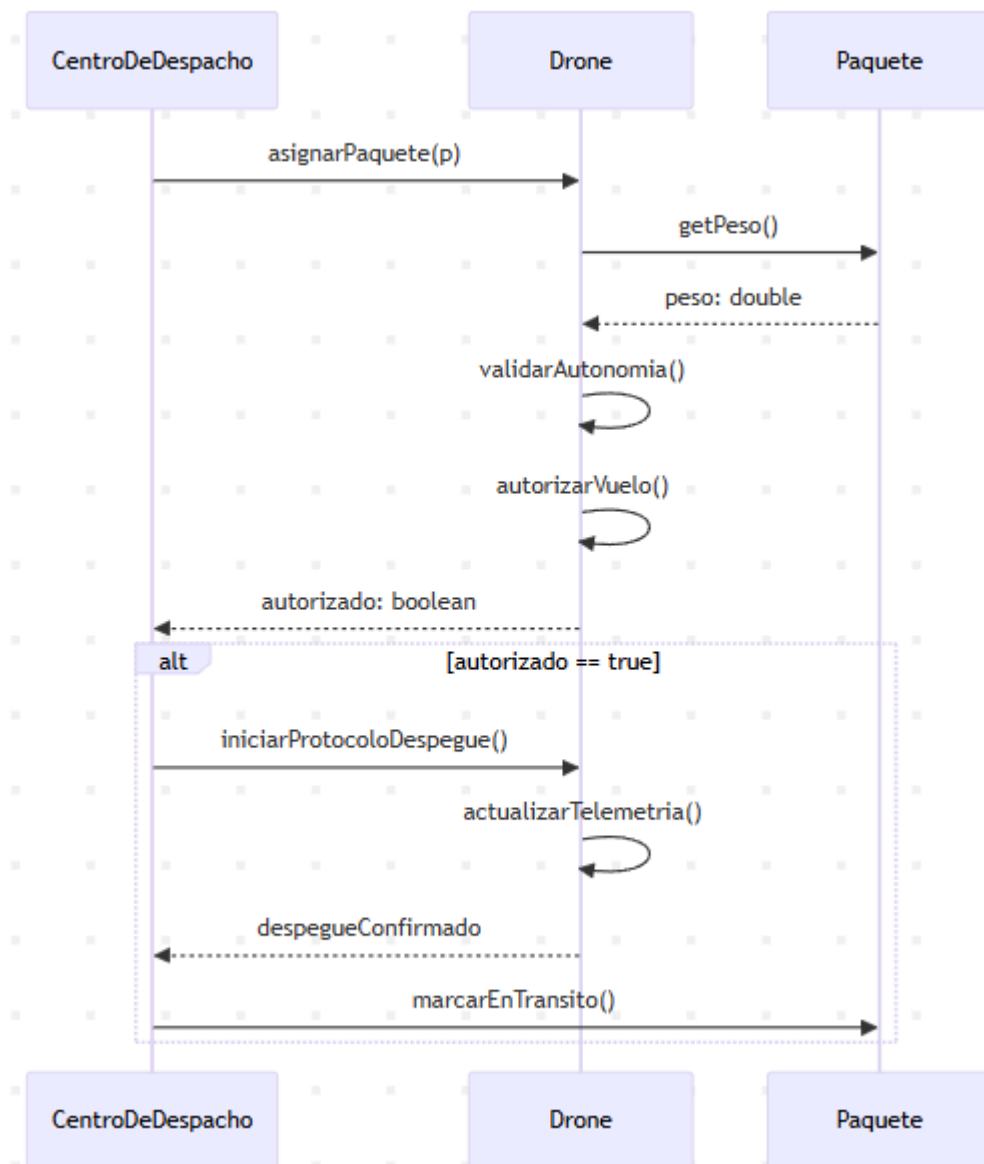
4.1 Escenario de Interacción Crítica

Modele el siguiente evento: "Un Centro de Despacho asigna un Paquete a un Drone; este válida su autonomía y, tras la confirmación, inicia el protocolo de despegue" .

4.2 Rigor en la Mensajería

El Diagrama de Secuencia debe respetar estrictamente el orden temporal y la interacción entre Centro de Despacho, Paquete y Drone. Se exige la inclusión de:

- Líneas de Vida y Activaciones (Barras de ejecución): Para representar el tiempo de control de cada objeto.
- Mensajes de Retorno: Para confirmar la finalización de las validaciones internas.
- Auto-mensajes: En caso de que el Drone realice validaciones internas de telemetría.



- El flujo comienza cuando el **CentroDeDespacho** envía el mensaje **asignarPaquete(p)** al Drone, estableciendo la relación entre el drone y el paquete a transportar. Después, el Drone solicita al Paquete su peso mediante el mensaje **getPeso()**, recibiendo este valor como respuesta para poder calcular el consumo de energía necesario.

- Luego, el Drone realiza una validación interna de su autonomía a través de un automensaje (**validarAutonomia()**), seguido de la ejecución del método **autorizarVuelo()**, donde determina si cuenta con suficiente batería para realizar el envío. Finalizado este proceso, el Drone devuelve al **CentroDeDespacho** el resultado de la validación, indicando si el vuelo está autorizado o no.
- Si la respuesta es positiva, el CentroDeDespacho envía la orden **iniciarProtocoloDespegue()**. El Drone, ejecuta un auto-mensaje para actualizar su telemetría (**actualizarTelemetry()**) y confirma el despegue. Finalmente, el CentroDeDespacho actualiza el estado del Paquete mediante el mensaje **marcarEnTransito()**.